

TUM Chair of Algorithms Project

Tear-aware Brushing for Map Tearing Analysis in DimBridge

Public short writeup - research project in progress

Executive summary

- We built a working prototype that connects map-tearing analysis from a MNIST embedding notebook to interactive brushing inside DimBridge.
- The system now marks top tear endpoints in the notebook, passes them into the DimBridge widget, and reports tear statistics for any brushed region.
- The current state is a validated prototype: the pipeline works end-to-end, but the project is still ongoing and the current results should be treated as preliminary.

1. Problem

Low-dimensional embeddings such as t-SNE or UMAP can visually separate points that are still close in the original high-dimensional space. These continuity violations are often described as map tears. The project goal is to make these tears easier to inspect interactively, rather than only ranking them offline in a notebook.

Concretely, we wanted to connect two components that were previously separate: (1) a notebook that detects top tear pairs on MNIST, and (2) DimBridge, an interactive visualization tool with brush-based selection.

2. What we implemented

- Selected the top 10 tear pairs from the MNIST t-SNE analysis.
- Converted those pairs into 16 unique tear endpoint points.
- Added a `tear_flag` column to the subset passed into DimBridge.
- Modified `ProjectionView.js` inside the forked DimBridge codebase.
- Integrated the tear check into `brush_end(event)`, so the computation happens after a brush selection is finalized.
- Logged tear-specific feedback in the browser console for each brushed region.

Pipeline

```
MNIST -> tear scoring notebook -> top tear pairs -> subset used in DimBridge -> tear_flag
injected into widget data -> brush interaction -> tear statistics reported for the brushed
selection
```

3. Relevant code snippets

Notebook-side preparation of the tear-aware data:

```
TOP_N_TEARS = 10
tear_points_global = []
for a, b in top_pairs_list_tsne[:TOP_N_TEARS]:
    tear_points_global.extend([int(a), int(b)])
tear_points_global = sorted(set(tear_points_global))
global_index_for_rows = np.array(idx_dimbridge).astype(int)
tear_flag_for_rows = np.isin(global_index_for_rows, tear_points_global).astype(float)
```

Widget-side data passed into DimBridge:

```
feature_cols = [c for c in df_dimbridge.columns if c not in ["index", "is_top_tear",
"tear_flag"]]
df_dimbridge_widget = df_dimbridge[feature_cols].copy()
df_dimbridge_widget["tear_flag"] = tear_flag_for_rows.astype(float)
dimbridge_tsne = Dimbridge(data=df_dimbridge_widget, x=Y_local[:, 0], y=Y_local[:, 1], ...)
```

Core logic added to ProjectionView.js:

```
let brushed_points = this.data.filter((d, i) => {
  return this.sample_brush_history.some((box) => {
    let px = this.x[i];
    let py = this.y[i];
    ...
    return px >= x0 && px <= x1 && py >= y0 && py <= y1;
  });
});
let brushed_tear_points = brushed_points.filter((d) => Number(d.tear_flag) === 1);
```

4. Current results

Notebook validation:

```
Top tear pairs used: 10
Unique tear endpoint points: 16
Tear points inside DimBridge subset: 16
```

Browser console validation before brushing:

```
[DimBridge tear-aware] first data row: {...}
[DimBridge tear-aware] tear column check: 16 out of 613
```

Browser console validation after a full brush:

```
[DimBridge tear-aware] Brushed points: 613
[DimBridge tear-aware] Tear points inside selection: 16
[DimBridge tear-aware] Tear density: 0.0261
[DimBridge tear-aware] Tear rows: [0, 29, 44, 60, 131, 208, 236, 247, 256, 301, 302, 326,
380, 512, 545, 584]
```

Interpretation: the full selection contained all 16 tear endpoints that were injected into the DimBridge subset. This validates that the tear information is correctly propagated from the notebook into the interaction layer.

5. What we have right now

- A forked and modified DimBridge codebase.
- A notebook that identifies top tear pairs and creates tear-aware metadata.
- A working tear-aware brushing prototype for MNIST.
- Console-level validation showing that selected regions can be checked for tear content.

At the current stage, the prototype is working and validated technically, but the project remains in progress. The present version is best described as an interactive research prototype rather than a finished public-facing product.

6. Immediate next directions

- Move from console-only feedback to visible tear overlays or tear summaries in the DimBridge interface.
- Run the same workflow on additional embeddings and datasets to assess whether the interaction generalizes beyond the current MNIST prototype.

Status note: this writeup is intended as a public short summary of ongoing work. Results are preliminary and subject to further validation.